machine learning

Felix Crisp

June 7, 2026

1 Introduction

Holography is a method of microscopy that requires no lenses and instead uses observed and tested principles from interference to reconstruct an image. At the universty of Kent a method using fiber optic cables has been developped in the hopes of it being used to access hard to reach locations in medical applications. However unlike with lenses, it's not possible to know where the focus length actually is, instead images a reconstructed through digital means at a range of depths, after that a user needs to search through these depths to find an image clear enough to be used. Ideally this process wouldn't be manual, currently 7 key metrics are built into the software, none perfectly predict the true depth of the image in focus. The goal then would be to find some kind of combination of the metrics avaible that, when searched for the lowest value, would reveal at which depth the an in focus image is. The latter part can be done really simply, the former though is more of a challenge.

To find this combination then, we will try using machine learning techniques, this application is quite natural for these techniques. The main one we will attempt to use is a linear regression using a decision tree in supposed to a classical method. The hope being that this should reveal some underlying pattern. In this goal the data was first analysed with basic methods, then simple attempts were made to do some kind of predictions before ending up on a concrete method that was then tested on larger scales.

2 Exploration and choices

2.1 Understanding the data

The data was prepared and pickled by Dr Michael Hugh. Thanks to this normalised data was already avaible allowing us to skip that step. The data was given as it's own class, it essentially ended up as a dictionary with several arrays inside. It was however, important to actually understand what this structure looked like and how to use it. With this in mind, the example code provided was ran and then adapted to a couple of other situations in order to see what changing some numbers did. From this I learnt how to manipulate the metric scores arrays using slices and how to properly load stuff. On top of the actual scores for each metric on each depth, the data loader function let me access some useful information like the true depth for each image. Other functions from the helpers file were useful but didn't end up making the final cut.

2.2 Exploring possibilities

Now that I could understand how to use the data I had to figure out what my goal metric would look like this. The first attempt was using the min_pos_error helper function. The idea was to train each metric independently (this is an incorrect approach and why is explained later) so that they could later vote on the right prediction. So each metric would be trained against the error between the depth of the minima of the metric and the true depth for that image. I also then had to choose a model to train, I had wanted to try a linear regression decision tree so that was the first attempt since a linear regression seemed like a good choice, and decision trees were a different avenue. In the end this model just didn't work.

The question is why. The awnser is that I didn't understand how voting systems actually work and why they work. They work by training a variety of models, that part would have been fine, however you can't use a specific subset for each of the models. This pushed me towards a different metric and using a multioutput regrressor unfortunately my plan having been to try and get a function which takes the original metrics as input and outputs a sort of idealised version of a metric (1 where it isnt in focus and 0 where it is), was short lived after learning that this method didn't actually work due to limitations of the implemtation of multioutput regressors in sklearn.

The problem then that I was really encountering is how to "squash the data" into a one dimensional metric. Since I couldn't train the regressors seperately and I couldn't train it to match a proper 2 dimensional function, I need to squash the goal into a 1D array. I ended up struggling a lot before being explained the idea of

essentially storing the depth you're at in the goal metric, what this ends up being is simply that for every single depth and every single image. we take the difference between the current depth and true depth and then apply the absolute value function. This now gives us quite the ideal metric to train against since it will be exactly 0 at the depth of the infocus image. This then led to training one decision tree with the input of an array with each value all 7 metrics and the output (what we wanted to predict), this new metric. This was the first big leap in quality of output.

3 Training and refining

3.1 Refining

An important part of a decision tree is it's max depth, so I decided that I should try and figure out the optimal depth to train the model at, to do this I decided to try running the model multiple times at the same depths since I noticed that depending on the tranining set, the quality of the model would change. This got quite taxxing as I was trying to train the same model at 12 different max depths, and repeat this nearly 100 times, so I implemented paralelisation using joblib, like this all 12 depths would train at the same time.

When increasing the number of jobs joblib could do at once by 3 times, I noticed a reduction in compute time of about 25%. In the end this meant running all 12 maxdepths at the same time each on their own thread, according to my own laptop's task manager this ended up using up about 50% of my cpu total, this is also including the baseline use that sits at about 15%. This told me that I could probably increase the amount of max depths I was testing, but as explained a few times through out this report, results were already inconsitent over iterations. This made me hesitant to try and change anymore of the codebase as quite a few values ended up hard coded to expect 12 max depths. This was a feature that the final code base attempted to fix but just ran out of time for the scope.

The entire reason for repeating the training multiple times was to determine the best max depth, in the end, it seemed that somewhere in the 5 to 8 range was most accurate, but that's the problem, the range was huge. I propose a few theories for this, first off a larger training set would deffinitely help, however that's only true if we get similar image classes. There's also lots of parameters that I didn't fully control yet, I started to use minimum leaf samples to help limit overfitting too, but some of my existing code base didn't adapt to it amazingly.

3.2 Final training

Using the admitedly inconsitent results from the previous part I could finally train my final model with a given max depth. Based on the results obtained from the paralelisation I chose to go with a max depth of 7. Then to evaluate how accurate these predictions would be I calculated the average prediction of the metric divided by the root mean square error. Depending on the image class or if I trained with all classes mixed I got a mean predicted value about 1.3 to 1.7 times bigger than the RMSE. I was quite dissapointed with these results.

4 Possible deployment and Lesson's learnt

4.1 Deployment

What would a deployment look like, first off it would have to include a consitent effort to keep improving the training set. The quality of the model would certainly increase for more specific tasks with higher volumes of data.

Something I think would be interesting is that when a true depth is recorded, an uncertainty could be added. Thinking back to how I've operated mircroscopes before, it's not always clear when something is deffinitely in focus, so adding the human uncertainty could give some insight. Since that would give us a target of accuracy, for instance if approximately 3 frames are in focus or 5. This isn't only useful as an indication of how accurate the model is, but could also server as a guide for how accurate the model needs to be. If the useful information can be gained from 3 different depths (contiguous ones) then we don't need to worry about an error of 0.01mm.

4.2 What I learnt

If I were to be given this task again there are a few things I would deffinitely change to my approach. First off I would have made document detailing naming conventions for some files and maybe even made a github for this specific task to help with management, whilst my orginisation was ok, I know it could be improved and I would have been a lot more efficient had I done that. Along side naming conventions, better code commenting could have helped a lot, especially for some of my weirder slicing or variable names. Secondly, I would create

a lot more basic graphs of the data to understand the structure better, in retrospect I kept running into issues because of my lack of understanding, that being said, I deffinitely learnt a lot about how to slice arrays and how effecient it can be. Lastly, I should have read more documentation. After strugling along with some of sklearn I spent an amount of time just reading it's documentation and as much as there was some weird stuff there, I gained a lot of insight into what was possible and what I should have been looking at.